code; in section 2.10 our challenge was finding the best branching factor for heaps used in Huffman coding.

---

[1] **A**pplication **P**rogramming **I**nterface

[2] Often bug-tracking suite associate lower numbers with higher priority. To keep things simple in our discussion, we will instead assume higher numbers mean higher priority).

[3] With an array implementation, we could find the right position for our item in logarithmic time, using binary search. But then we would need to move all the elements on the right of the insertion point to make room for it, and this requires linear time on average.

[4] Besides performance, there are other aspects we might need to check, depending on our context. For instance, in a distributed environment, we must make sure our implementation is thread-safe, or we will incur in race conditions, the nastiest possible bugs that could afflict our applications.

[5] You can find an introduction to algorithm analysis and big-O notation in appendix B.

[6] If a problem is really tractable or not for a certain size of the input also depends on the kind of operations performed and on the time they take. But even if each operation takes as little as 1 nanosecond, if the input has 1 million elements, a quadratic algorithm will take more than 16 minutes, while a linearithmic algorithm would require less than 10 milliseconds.

[7] A leaf is a tree node that doesn't have any children; an internal node is a node that has at least one child or, equivalently, a node that is not a leaf.

[8] We use explicit parenthesis for the following expressions. In the rest of the book, we will generally omit redundant parenthesis, so that, for instance, we will write `2*i + 1`

[9] $(a_1, b_1, c_1,\ldots) < (a_2, b_2, c_2,\ldots)$ if and only if $a_1 < a_2$ or ($a_1 == a_2$ and $(b_1, c_1,\ldots) < (b_2, c_2,\ldots)$)

[10] The branching factor of a tree is the maximum number of children a node can have. For example, a binary heap, which in turn is a binary tree, has branching factor 2. See appendix C for more details.

[11] As we explain in appendix C, this doesn't change the asymptotic analysis, because , it can be proven that inserting n elements in a dynamic array requires at most 2n assignments, so each insertion requires an *amortized* constant time.

[12] For instance, in Java's `PriorityQueue`, a class provided in standard library, you need to remove an element and then insert the new one to perform the same operation. This is far from ideal especially because the removal of a random element is implemented inefficiently, requiring linear time.

[13] Using Fibonacci heaps, it could be implemented even in amortized constant time, at least in theory.

[14] Usually, though, the greatest concern is to avoid choosing the wrong data structure…